

FLOW FORM

Trabajo Práctico Especial

Nombre	Apellido	Legajo	E-Mail
Diego	Badin	63551	DIEGO BADIN
Diego	Rabinovich	63155	DIEGO JOSÉ RABINOVICH
Magdalena	Taurian	62828	MAGDALENA TAURIAN

24 DE NOVIEMBRE DE 2024

ITBA

Tabla de Contenidos

1. Introducción	2
2. Modelo computacional	2
2.1 Dominio	2
2.2 Lenguaje	2
2.2.1 Inicio del formulario	3
2.2.2 Configuración del formulario	3
2.2.3 Condición de visibilidad. @ShowIf.	3
2.2.3.1 @ShowIf anónimo.	3
2.2.3.1 Declaración de @ShowIf .	4
2.2.3.1 Llamado de @ShowIf .	4
2.2.4 Organización dentro del formulario. Definición de Steps y Sections	4
2.2.5 Definición de Questions	5
3. Implementación	6
3.1 Frontend	7
3.2 Backend	7
3.2.1 Llenado de tablas	8
3.2.2 Verificación de integridad de datos	8
3.2.3 Generación de código	9
3.3 Adicionales	9
3.4 Dificultades Encontradas	15
4. Futuras Extensiones	15
5. Conclusiones	16
6. Referencias	16
7. Bibliografía	16

1. Introducción

FlowForm es un compilador que fue confeccionado utilizando herramientas de análisis léxico, sintáctico y semántico, que reconoce un lenguaje que lleva el mismo nombre, y que fue diseñado de manera verbosa, anteponiendo la facilidad de uso.

El presente documento explora la idea detrás del compilador presentado, el problema que busca resolver, su definición, el dominio, así como también repasa los ejes principales de su implementación.

Si bien sólo se trata de una primera versión, el lenguaje acepta buena cantidad de las condiciones más frecuentes en la confección de formularios inteligentes, la división de preguntas en pasos, los pasos en secciones y la posibilidad de armar formularios sencillos, simples o complejos y completos.

2. Modelo computacional

2.1 Dominio

FlowForm es un Lenguaje de Dominio Específico (DSL por sus siglas en inglés). El problema del que se ocupa es de la generación de formularios que contienen lógica condicional. Los formularios son una parte importante de la lógica de negocios de muchas páginas y aplicaciones web, así como también son un punto importante a la hora de realizar encuestas para obtener un feedback o bien para facilitar el guardado de información.

En este sentido hay una necesidad recurrente de complejizar el formulario para otorgarle mayor personalización al mismo. Esto se traduce en el agregado de pasos, secciones, validaciones de entradas, etc. Pero aún con toda esa personalización que es relativa a la popularización y validación del formulario, es probable que se desee complejizar aún más el mismo, para darle lugar a que por ejemplo el formulario admita visibilidad condicional de preguntas, basado en respuestas anteriores. Yendo incluso más allá, es posible que se quiera restringir la visibilidad de algunas opciones en campos de selección. Para solucionar este problema se provee esta primera versión de un compilador que interpreta un lenguaje diseñado para crear un formulario multiuso.

FlowForm genera un archivo HTML que contiene todo lo necesario para que al abrirlo sea un formulario con estilos predefinidos inspirados en Taylor Swift. Este archivo provee una función que permite al usuario modificarla para decidir qué hacer con el submit del formulario. De esta manera se le podrá dar múltiples usos al mismo.

Además FlowForm provee la posibilidad de generar un archivo GS que sirve para que en pocos clics un usuario con una cuenta de Google pueda deployar el formulario y recibir las respuestas en un google spread sheet.

2.2 Lenguaje

FlowForm tiene una estructura dividida en lo que denominamos “Fragmentos” (comienzan con @) y “Especificadores” (comienzan con #). Los fragmentos respetan una jerarquía, mientras que los especificadores son atributos del objeto que define el fragmento. La diferencia entre ambos radica en el nivel de capacidad para albergar “código libre” o condiciones.

Los símbolos son globales. Por lo cual está prohibido tener dos identificadores iguales aún si son de distinto tipo o están en diferentes scopes. Esto se debe en parte a que las condiciones en un formulario suelen hacer referencia a preguntas a lo largo del mismo. Entonces es importante que una condición

referencie sin ambigüedades a que identificador se refiere. Cuando se referencia el id de una pregunta, esta tomará el valor de la entrada del campo del formulario.

El lenguaje ofrece las siguientes funcionalidades:

2.2.1 Inicio del formulario

Se podrá definir el título del formulario con el atributo **#Title**, su descripción con **#Description** y un mensaje de salida en la terminación del formulario con **#Closure**.

```
#Title "The Form Title"
#Closure "Thanks for submitting the form"
#Description "The form description"
```

Código 1: Headers del FlowForm.

2.2.2 Configuración del formulario

Se podrá configurar el output del programa y el estilo del formulario mediante la declaración **@FormConfig**.

- Se configura que las respuestas del formulario se almacenen en un Google Spreadsheets al agregarle el atributo **#SafeAndSound**.
- Se especifica el texto dentro del botón que envía el formulario con el atributo **#SubmitBtnText** y a continuación el texto.
- Se puede seleccionar un diseño de los provistos por la librería estándar de FlowForm. Esto se consigue agregando el atributo **#Theme** seguido del tema deseado. La lista de temas deseados son: `_Debut`, `_Fearless`, `_SpeakNow`, `_Red`, `_1989`, `_Reputation`, `_Lover`, `_Folklore`, `_Evermore`, `_Midnights` y `_TTPD`. De no especificar se utilizara el diseño predeterminado.

```
@FormConfig {
    #SubmitBtnText "Submit"
    #SafeAndSound
    #Theme _TTPD
}
```

Código 2: Ejemplo de un @FormConfig completo.

2.2.3 Condición de visibilidad. @ShowIf.

Para establecer la visibilidad de una sección o pregunta, se puede utilizar el fragmento **@ShowIf**. Nótese que sólo es un fragmento válido si se encuentra dentro de un **@Question** o de un **@Section**.

Existen tres distintos tipos de escribir un fragmento de este tipo. Por un lado están los de “uso” que son “anónimo” (ver 2.2.3.1) o “llamado” (ver 2.2.3.3), de los cuales solo se acepta uno por fragmento (de haber más no avanzara a etapa de generación de código). Y por otro lado está la “declaración” (ver 2.2.3.2), que se aceptan múltiples por fragmento.

2.2.3.1 @ShowIf anónimo.

Entre llaves se declara la condición, de estilo lógica, a cumplir para decidir si la sección o la pregunta se muestra en el formulario. Se hace de manera anónima, y afecta sobre todo el fragmento de sección o de pregunta en el que se especificó. Solo se acepta un **@Showif** anónimo o un “llamado” (ver 2.2.3.2) por fragmento.

```
@ShowIf { howOld isGreaterThan 18 }
```

Código 3: Ejemplo de un @ShowIf anónimo .

2.2.3.1 Declaración de @ShowIf .

También se puede declarar **@Showif** para utilizar en el futuro en cualquier parte del formulario. La condición no afecta el fragmento de sección o pregunta donde se la define. Solo es la declaración.

```
@ShowIf isOlderThan18 { howOld isGreaterThan 18 }
```

Código 4: Ejemplo de una declaración de @ShowIf.

2.2.3.1 Llamado de @ShowIf .

Luego se puede utilizar el condicional que debe estar previamente declarado para configurar la visibilidad de la sección o de la pregunta.

```
@ShowIf(isOlderThan18)
```

Código 5: Ejemplo de un llamado a un @Showif previamente declarado.

2.2.4 Organización dentro del formulario. Definición de Steps y Sections

Se podrá organizar el formulario en pasos y secciones mediante las declaraciones **@Step** y **@Section** que deben ir dentro de los steps, de haber en el formulario.

Dentro de estos apartados se puede especificar el título con el atributo **#Title** y se puede agregar una descripción con **#Description**.

Dentro de ambas partes del formulario se pueden definir preguntas con la declaración **@Question**. Con preguntas nos referimos a las consignas que tiene que completar el usuario al completar el formulario. El único requisito es que se mantenga la jerarquía observada en “Código 3”

```
#Title "Form Title"
@Step stepIdentifier {
  #Title "Step Title"
  @Section {
    #Title "Step Title"
    @Question questionIdentifier {
      #Title "Question Title"
    }
  }
}
```

Código 6: Jerarquías dentro del formulario.

Es decir, que si se encuentra el padre tiene que respetarse la jerarquía. Por ejemplo, si el step tiene secciones, todas las preguntas van a estar dentro de las secciones. En el caso de que no haya secciones en el step, se pueden definir preguntas directamente, o secciones. De esta manera lo que se prohíbe es la existencia de preguntas, secciones o steps a la misma altura de jerarquía.

Además los **#Title** son **obligatorios** cuando el fragmento aparece. Y un formulario sin ningún fragmento de pregunta, es invalido, y fallará antes de comenzar la generación de código.

Se puede definir dinámicamente el flujo del formulario con la declaración **@GetawayCar** que especifica a qué step deberá ir al completar el actual o si se debería enviar el formulario. Se especifica al agregar luego de un **WHEN** la condición que se debe cumplir para el salto y la declaración **GOTO** seguida del nombre del step o **END** si se debería terminar el formulario.

```
@GetAwayCar {
    WHEN howOld isLowerThan 13 GOTO END
    WHEN howOld isGreaterThan 18 OR howOld isEmpty GOTO s3
}
```

Código 7: Ejemplo de un @GetAwayCar.

2.2.5 Definición de Questions

Se podrán definir consignas para que se completen en el formulario con la declaración **@Question**.

- Se determina el tipo de input que se recibe con el atributo **#Type**. Las opciones posibles son: CHECKBOX, RADIOS, SELECT TEXT, SELECT NUMERIC, SELECT DATE, TEXT, IMAGE, DOCUMENT, LONGTEXT, NUMERIC, PASSWORD y DATE. Si el especificador **#Type** no se define, entonces se asume que la pregunta es tipo TEXT.
- Se agrega la consigna a completar por los usuarios del formulario con **#Title**.
- Se puede especificar cuándo debe visualizarse una pregunta con el fragmento **@Showif**. Que puede ser declarado anónimamente (como con las secciones) o un llamado a una previamente declarada.
- Se pueden hacer declaraciones de **@Showif**, las mismas no afectan a la condición de visibilidad de la pregunta. Solo se declara para usar en el futuro en cualquier parte del formulario.
- Con el atributo **#PlaceHolder** se puede agregar un ejemplo del input que se espera recibir en la consigna.
- Se puede asignar un valor inicial a la pregunta con el atributo **#Default** seguido de dicho valor.
- Se puede especificar si la pregunta es obligatoria o no con el atributo **#Required**.
- Si se cree necesario agregar información en la consigna, con el atributo **#Help** se puede agregar un texto que se visualiza al hacer un hover sobre el icono de información que se localiza a la derecha del **#Título** especificado.
- En ciertos Types se requieren opciones y se pueden especificar con el atributo **#Options** y las opciones especificadas a continuación separadas por comas y entre corchetes. Las opciones se pueden modificar ya que se puede agregar un **@ShowIf** para cada una de ellas. Como se puede observar en “Código 8” solo muestra la opción “Heineken” si en la pregunta howOld, la respuesta fue mayor a 21. También se puede hacer un llamado a un **@Showif** previamente declarado, con el mismo efecto.

```
@Question favoriteBeer {
    #Type SELECT TEXT
    #Title "What's your favorite beer?"
    #Options ["Brahma", "Patagonia", "Heineken" @ShowIf { howOld isGreaterThan
21 }]
}
```

Código 8: Ejemplo de un @ShowIf en una #Option en particular.

Se puede restringir la respuesta del usuario con la declaración **@Glitch**. Dentro de esta declaración se pueden especificar varias situaciones de error con **@Error**. Esta especifica la condición en la que debería marcar el error con un **@ShowIf** y con el atributo **#Message** se aclara al usuario por que su respuesta no es válida. También es posible hacer un llamado a un **@Showif** previamente definido.

```
@Glitch {
  @Error {
    #Message "Too long"
    @ShowIf { name isLongerThan 20 }
  }
  @Error {
    #Message "Too short"
    @ShowIf { name isShorterThan 1 OR name isEmpty }
  }
}
```

Código 9: Ejemplo de un @Glitch.

3. Implementación

En esta sección se va a indagar en el proceso de creación de FlowForm.

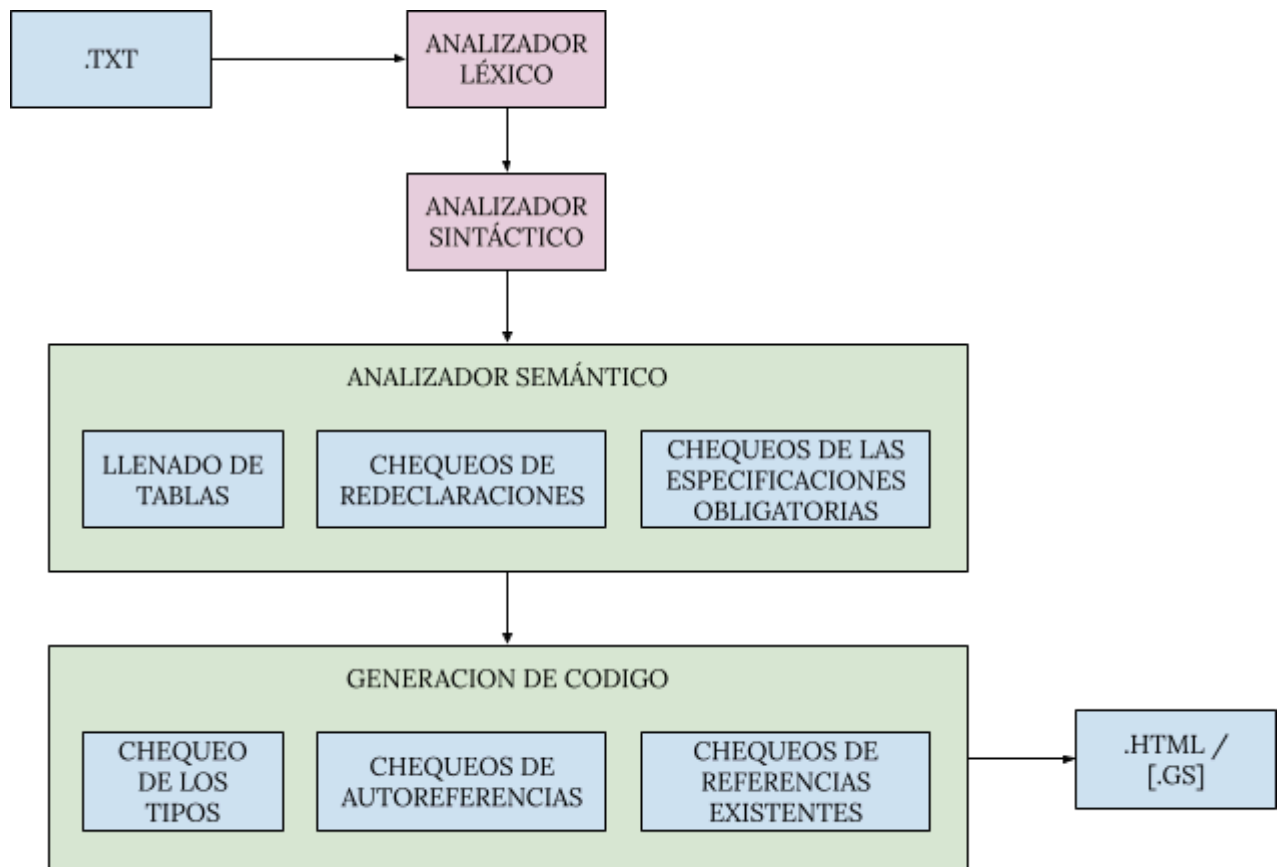


Figura 1: Flujo del desarrollo del lenguaje y compilador.

3.1 Frontend

En este apartado se utilizaron como herramientas principales el analizador sintáctico de Bison y el analizador léxico Flex. Dado que la definición del lenguaje sigue una estructura de jerarquías previamente definidas el desarrollo de las funciones implementadas para la construcción de los nodos que forman parte del árbol sintáctico generado no generó grandes complicaciones.

Dado que el lenguaje utiliza una sintaxis propia se optó por el diseño de jerarquías anidadas lo que simplificó la definición de las producciones utilizadas y lo que permite una identificación temprana de formularios mal formados por mala jerarquía.

Se definieron estructuras y enumerados específicos para realizar el manejo de datos, operaciones, e identificación de tokens, contemplando las diferentes variaciones en la construcción del formulario. Las estructuras fueron diseñadas como se observa en el código 10, utilizando uniones para optimizar el uso de memoria y tipos de datos previamente definidos para mejorar la organización del árbol.

```
enum Types {
    TYPE_STRING,
    TYPE_FLOAT,
    TYPE_INTEGER,
    TYPE_DATE,
    TYPE_ID,
    TYPE_NONE,
    TYPE_VALUE,
    TYPE_NUMBER,
    TYPE_TRUE,
    TYPE_FALSE
};

struct Date {
    int day;
    int month;
    int year;
};

struct Value {
    union {
        char * v_string;
        int v_integer;
        float v_float;
        Date * v_date;
    };

    Types type;
};
```

Código 10: Ejemplo de estructura de datos definida.

3.2 Backend

Una vez que con las acciones de bison tenemos generado el árbol de sintaxis abstracto, damos comienzo al backend del proyecto. El mismo se dividió en 3 partes, primero se recorre el árbol para completar tablas,

luego se hace un recorrido por las tablas principales para verificar que el estado sea lo suficientemente bueno como para continuar hacia el tercer paso: la generación de código.

Los tres pasos son independientes entre sí. El compilador imprimirá todos los errores encontrados e intentará continuar con la ejecución lo máximo posible, para notificar cuantos errores pueda al usuario. No obstante de encontrar errores previo a la tercera etapa (generación de código) no se avanzará con la misma. Si hubiera errores detectados durante la generación de código, de igual manera se intentará avanzar todo lo que se pueda, para imprimir todos los errores, y luego se borrará la salida generada.

3.2.1 Llenado de tablas

El primer paso del backend es navegar el árbol para completar las distintas tablas que albergarán toda la información necesaria, en un formato conveniente para la generación de código. Por la naturaleza del lenguaje, es importante verificar que no se re-declaren especificaciones o condiciones. Hay que tener en cuenta que si bien para los fragmentos que tienen identificadores, como `@Question`, `@Step`, o las declaraciones de `@Showif`, es posible utilizar la tabla de símbolos (que en realidad es un mapa hash [\[1\]](#)) para verificar que no esté declarada previamente, no es el caso de los demás especificadores o fragmentos, que son anónimos. Para estos casos recurrimos a tablas, que además de guardar la información completa de una pregunta, de las opciones, los getaways, las secciones, los pasos, etc. Sirven para saber si ya estaban presentes esos atributos o fragmentos en el fragmento padre. Así es que es posible chequear si un `@Question` contiene una re-declaración de un `#Title`, imprimir el error y avanzar.

No obstante incluso con las tablas algunos especificadores son especialmente complicados de validar, pues el valor nulo, o cero puede ser opción y no necesariamente implica que no hubiera sido declarado con anterioridad. Para esto se opta por usar unas pequeñas estructuras globales de “flags” o banderas, que brindan información sobre el estado actual, y lo que ya fue encontrado.

Un caso particular es el de las condiciones, sean `@Showif` o `@Getaways`, es muy preferible que el generador de código trate una condición como un subárbol en lugar de una tabla. Evidentemente una expresión lógica puede crecer mucho, y es muchísimo más manejable siendo un árbol binario. Es por eso que las condiciones no son expandidas en esta etapa. Y además de querer expandirlas en esta etapa deberíamos primero hacer una pasada para reconocer todos los símbolos que podrían llegar a aparecer en las condiciones, ya que estas pueden referenciar símbolos que aún no hayan sido registrados.

3.2.2 Verificación de integridad de datos

Una vez formadas todas las tablas, se verifica que los requerimientos mínimos de cada tabla se cumplan. Esto valida que por ejemplo todas las preguntas tengan definido un título, lo mismo con las secciones, los steps, el formulario, también se verifica que exista al menos una pregunta.

Al finalizar esta fase se imprime a modo de información todo lo que el compilador encontró (ver *figura 2*). No obstante, esta fase no garantiza que ya no haya errores.

```
[INFO ][FlowForm] Form Configuration set
[INFO ][EntryPoint] Computed 3 steps successfully
[INFO ][EntryPoint] Computed 2 sections successfully
[INFO ][EntryPoint] Computed 2 getaways successfully
[INFO ][EntryPoint] Computed 3 options successfully
[INFO ][EntryPoint] Computed 1 showIf declarations successfully
[INFO ][EntryPoint] Computed 3 glitches successfully
[INFO ][EntryPoint] Computed 7 questions successfully
[INFO ][EntryPoint] Computed 11 symbols successfully
```

Figura 2: Fin exitoso de la fase de verificación.

3.2.3 Generación de código

Para generar el código, en primer lugar se copian al archivo de salida del HTML los headers y las funciones en javascript de las ofrecidas en la librería estándar de flowform, que son las que usan las condiciones (aunque algunas condiciones se resuelven sin necesidad de una función js). Luego se recorren las tablas para armar los arreglos de preguntas y pasos con toda la información necesaria para que incluso los casos más complejos puedan resolverse sin dificultad, gracias a un paquete prediseñado de funciones que se encargan de interpretar estos arreglos y seguir las lógicas descritas. Para esto es necesario considerar que si un formulario no tiene pasos, de igual manera se agrega uno vacío, a fines de que sea la base. Si un paso no tiene secciones, se incluye una única sección vacía, en la que irán a parar todas las preguntas de ese paso. Y así se puede seguir con la combinatoria de casos.

Además es en esta etapa en la que las condiciones se expanden, por primera vez. Entonces es posible encontrar errores del estilo de identificadores no definidos, o uso de parámetros con tipos no válidos para las funciones. Por ejemplo, si una función comparaba `q1 >= "árbol"` es necesario verificar que `q1` sea un identificador de una pregunta que tenga un tipo numérico, y de igual manera que `"árbol"` sea en realidad un número. Si se da un caso como este, el compilador imprime el error, marca que algo salió mal y avanza, no obstante es posible que debido a que se continúa con la generación luego nos encontremos errores que quizás no son tan precisos, y son desencadenados por el anterior. Al igual que en cualquiera de las otras fases.

Para la generación de código se utiliza incorporado en un mismo archivo el framework "Vue [2]" y "Vuetify [3]", para facilitar el manejo de las condiciones haciendo uso de variables reactivas.

Por otro lado en esta etapa nos es posible validar fácilmente si hay "autorreferencia" en una condición de getaway, lo que haría que sea imposible avanzar hacia la siguiente sección.

Si durante la generación de código se encuentran errores, el código generado se descarta, al final.

3.3 Adicionales

Adicionalmente se implementó un atributo para poder almacenar las respuestas del formulario en un Google Spreadsheet al generar además del .HTML un archivo .gs.

A continuación se mencionan los pasos a seguir para su correcto funcionamiento:

- 1) Dirigirse a <https://script.google.com/home>
- 2) Iniciar sesión o crearse una cuenta.
- 3) Hacer click en "New Project".
- 4) Hacer click en "Add file" y "Script". Copiar el archivo *"Form.gs"* generado por el compilador FlowForm.



Figura 3: Paso (4) de la configuración del SpreadSheet.

- 5) Hacer click en "Add file" y "Script". Copiar el archivo *"Form.html"* generado por el compilador FlowForm.
- 6) Redefinir las variables a gusto en el archivo *"Form.gs"* y guardar:

```
var SPREADSHEET_NAME = 'Form Answers';
var SHEET_NAME = 'Answers';
```

Código 11: Variables que se pueden redefinir del "Form.gs".

Aclaración importante: el parámetro en el return del doGet() tiene que coincidir con el nombre del archivo .html

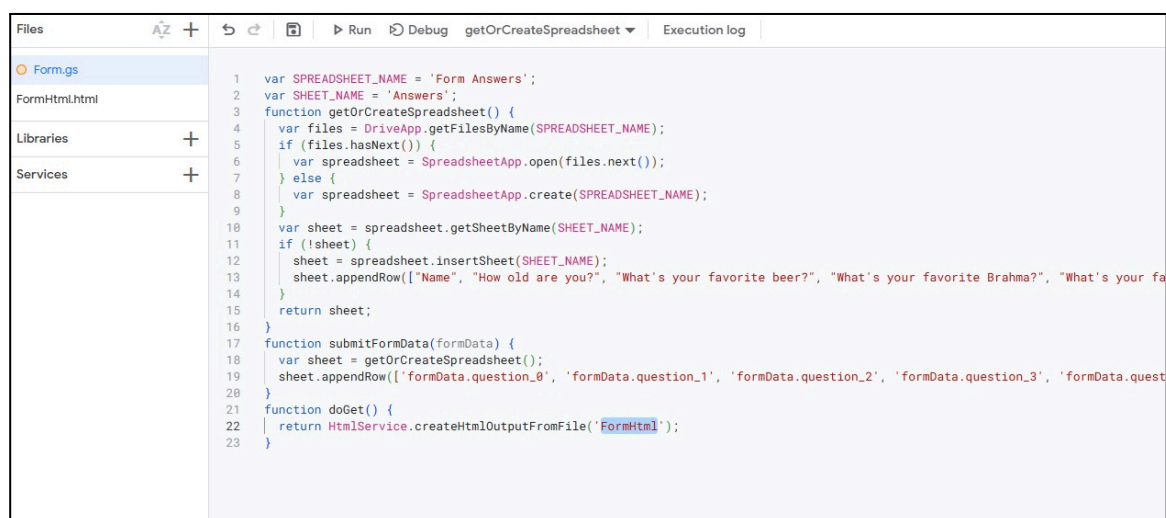


Figura 4: Para la configuración del SpreadSheet el nombre en el parámetro seleccionado en la línea 22 tiene que coincidir con el nombre del .html.

- 7) Hacer click en "Deploy" y "New deployment".
- 8) En la sección de "Select type", hacer click en el engranaje de configuración y en "Web app".

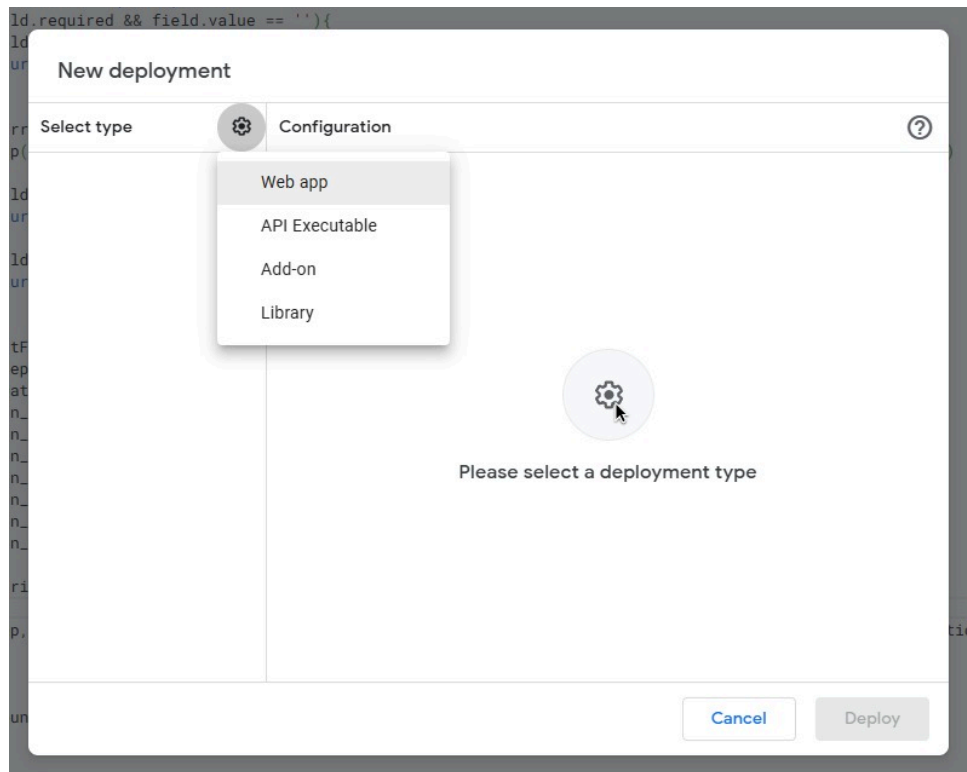


Figura 5: Paso (8) de la configuración del Spreadsheet.

La configuración consta de una descripción breve del deploy, y quién tendrá acceso al mismo. Se puede setear para que solo vos, usuario, tengas acceso, o cualquiera con o sin una cuenta de Google.

Por el momento no es posible restringirlo por mails o dominios (a menos que el usuario sea un developer de google que haya abonado la fee).

- 9) Especificar en “Who has access” quienes se quiere acceder al formulario.

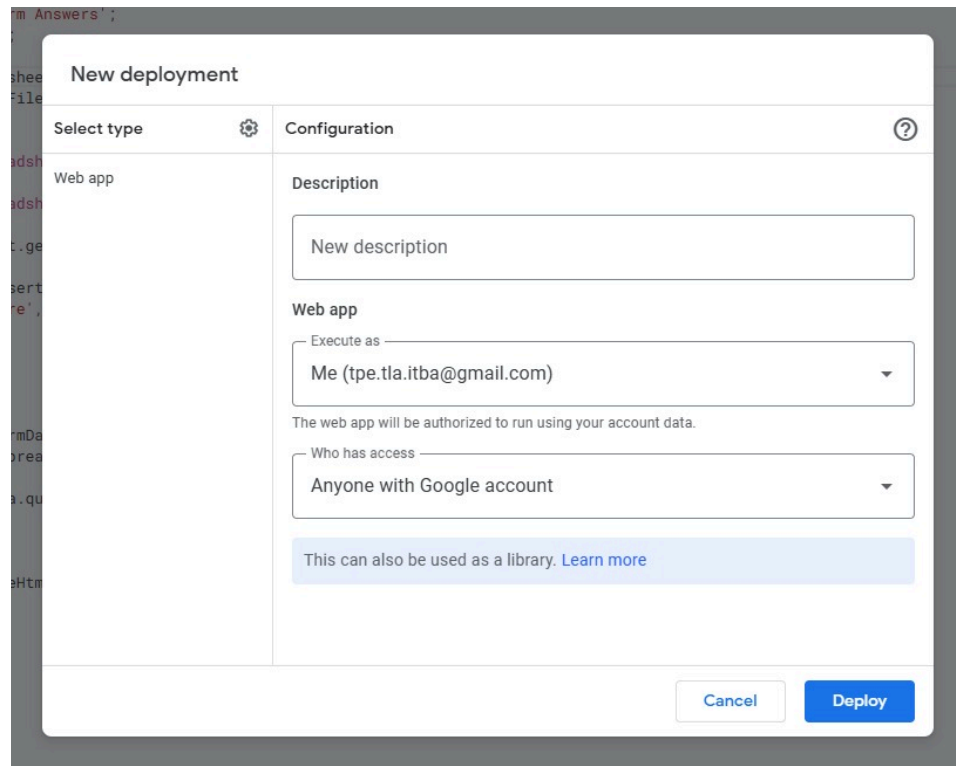


Figura 6: Paso (9) de la configuración del Spreadsheet.

10) Hacer click en "Deploy" y "Authorize access".

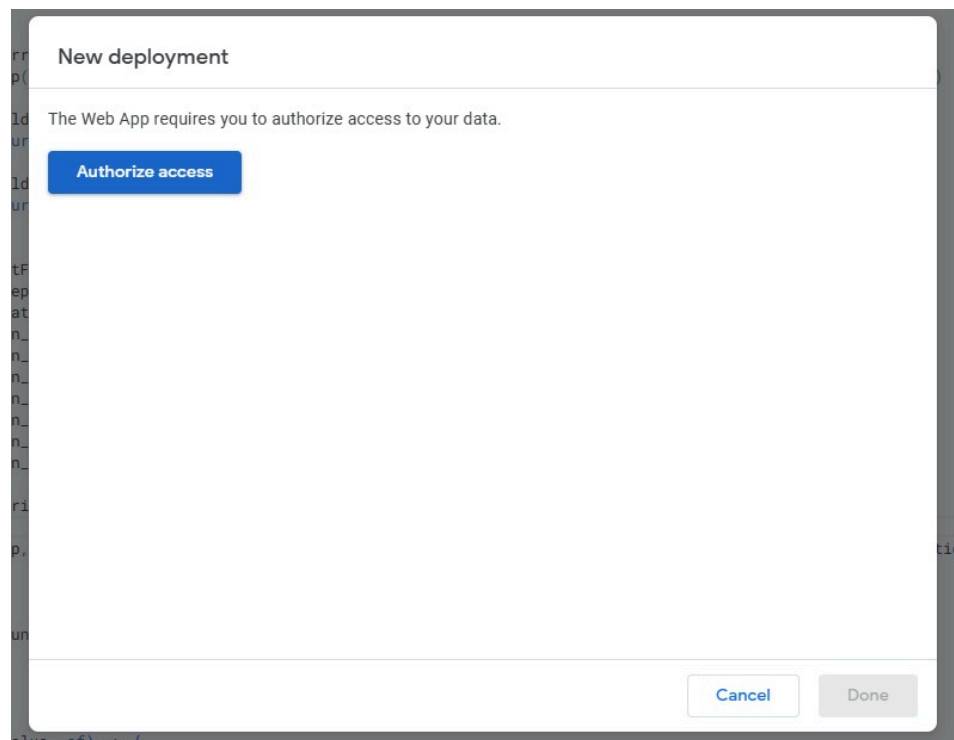


Figura 7: Paso (10) de la configuración del Spreadsheet.

11) Ingresar dirección de mail y hacer click en "Advanced"

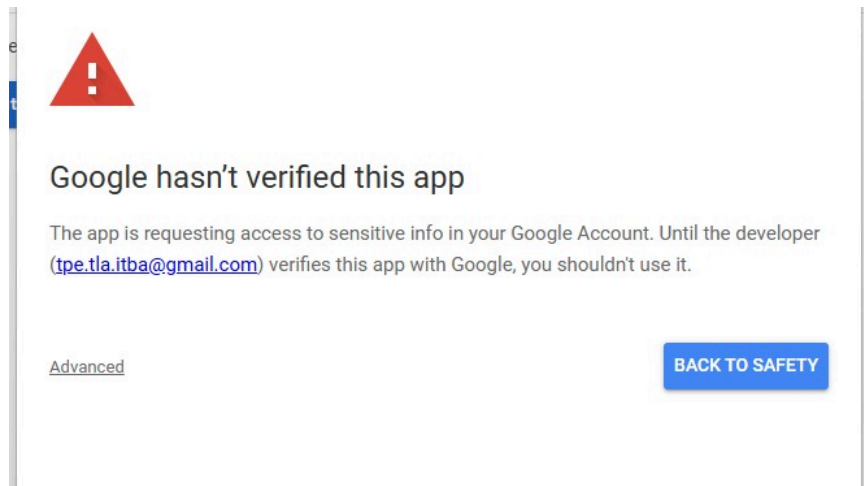


Figura 8: Paso (11) de la configuración del SpreadSheet.

12) Hacer click en "Go to [project name]".

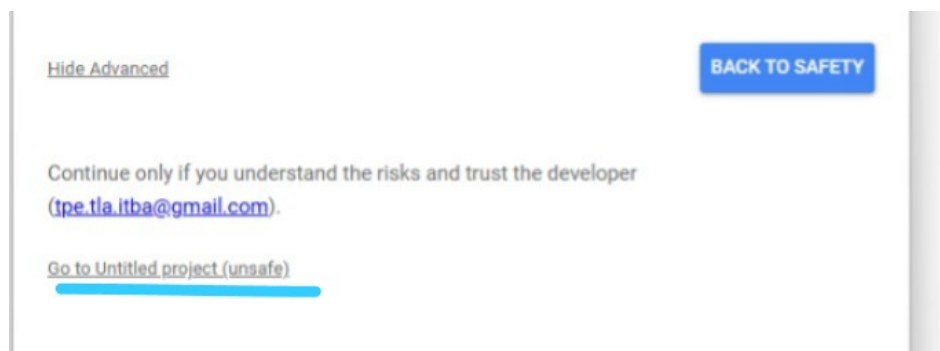


Figura 9: Paso (12) de la configuración del SpreadSheet.

13) Hacer click en "Allow".

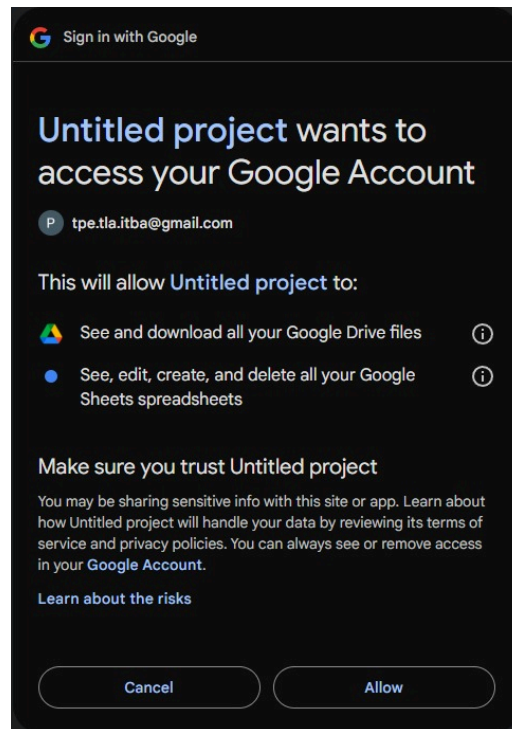


Figura 10: Paso (13) de la configuración del Spreadsheet.

- 14) Una vez deployado se tendrá un link de scripts de Google totalmente gratuito con el formulario en la Web. Las respuestas se cargarán en la hoja de cálculo de Google que se guarda en el Google Drive, con el nombre "FlowForm_Form_Answers" o el que se haya elegido.

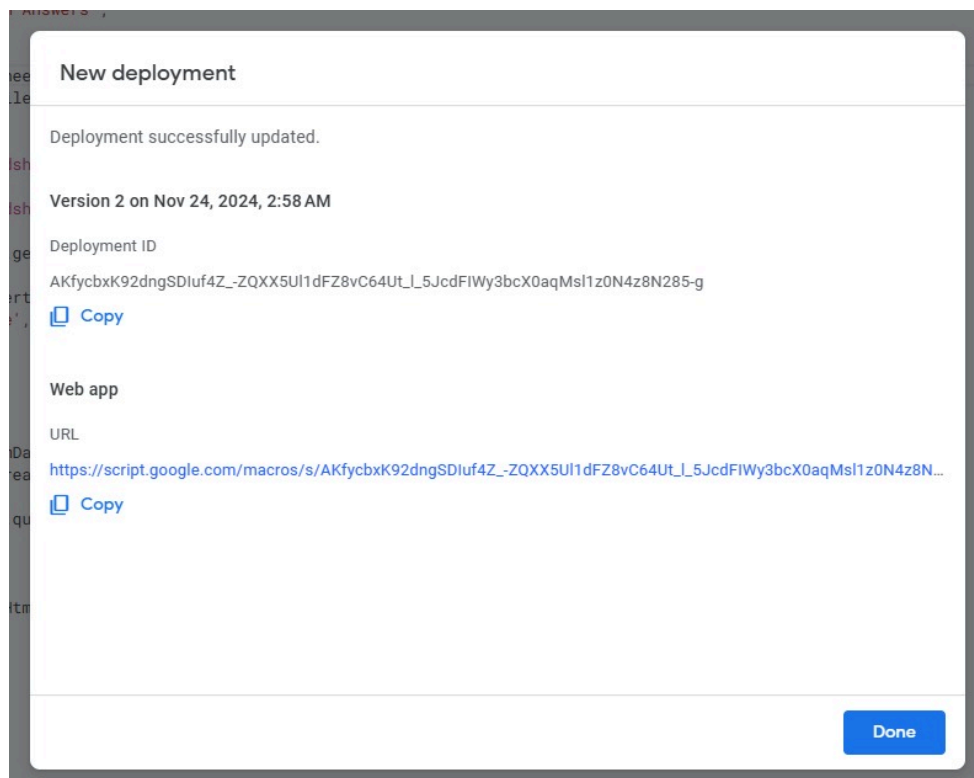


Figura 11: Paso (14) de la configuración del Spreadsheet.

Aclaración importante: si el documento ya existe la hoja se agrega al mismo. Y si la hoja ya existe, se añade la respuesta en la primera fila vacía.

Formulario ejemplo proveniente del test (disponible en el repositorio) de aceptación

“full_form_3_steps.txt”:

https://script.google.com/macros/s/AKfycbycRfNk1c_T5Su4rEd2W5tZwecUsjOQ7eIXNpGnEC96r1Pyh7RkBTZV-45sNAGQB0FGfA/exec

Aclaración importante: El flujo descrito en la sección fue realizado con una versión de la web válida el día 24/11/2024, el detalle de los mismos queda sujeto a las variaciones de la interfaz en un futuro.

3.4 Dificultades Encontradas

En lo referente al diseño del lenguaje, nos encontramos con varias dificultades. Diseñar un lenguaje es irónicamente difícil pues como desarrolladores, lo más común es utilizarlos pero no es tan común sentarse a pensar cómo debería ser el lenguaje ideal para resolver un problema dado. En este sentido nos dimos cuenta que el diseño que propusimos no es el que más nos hubiera gustado, y contiene algunas sintaxis innecesarias, como es el caso de `@Glitch` que al final termina siendo solo un array de `@Error`. Por lo que podría retirarse `@Glitch`. Otro ejemplo es el caso de `@ShowIf`, quien no puede declararse en `@Steps`, ni como declaración ni llamado ni de forma anónima. En un principio le encontramos justificación, pero finalmente creemos que debería ser agregado en un futuro. No obstante, en vistas generales estamos conformes con el lenguaje que diseñamos.

En lo referente a la implementación de frontend, no tuvimos demasiadas problemáticas.

En lo referente al backend, nos encontramos con la dificultad de manejar todos los casos de error, y avanzar intentando no “explotar”. Se tomaron decisiones que creemos acertadas, que permiten que el compilador avance todo lo que pueda, pero como consecuencia podrían darse casos de error imprecisos. Por ejemplo si se encontrase un identificador de pregunta duplicado, es posible que luego se determine que una pregunta no tiene un requerimiento obligatorio como “#Title” cuando en realidad quizás si lo tiene. Este tipo de problemas son comunes en compiladores conocidos. Todos hemos visto como un “,” que no está puede terminar marcando un supuesto error inexistente en otro lado, trabajando con C. Si bien pueden darse casos como este, creemos más valioso para el usuario que logre ver la mayor cantidad posible de errores antes de tener que volver a compilar.

4. Futuras Extensiones

Como próxima extensión a desarrollar, se buscaría poder ampliar los casos de error de la autorreferencia. Actualmente solo se tiene en cuenta si en un `@GetAwayCar` se hace un GOTO a sí mismo. Por ejemplo, se tiene como objetivo también que no dependa del input propio para su aparición. Es decir, que el `@ShowIf` propio no haga referencia a sí mismo. Este problema podría ser indecidible, no obstante podríamos aplicar heurísticas para avisarle al usuario acerca de posibles autoreferencias.

Además los checkboxes y los radio buttons no se visualizan de la forma deseada por el equipo. Estos no se ven a simple vista solamente si se hace hover sobre ellos.

Por otro lado, y yendo más allá estaría muy bueno que el código no sea solo de front, y provea una salida de código de back para validar el formulario del lado del servidor con todas las condiciones. Esto no sería tan difícil de hacer en el .gs por lo menos para el caso de encuestas. Pero para el scope del cuatrimestre no dio tiempo. También estaría bueno que pueda elegir en qué lenguaje o framework quiere la salida el usuario. Para que no sea solo JavaScript y se acepte TypeScript por ejemplo.

5. Conclusiones

En este informe se detalló el desarrollo de un compilador para la generación de formularios en formato HTML mediante un lenguaje de dominio específico (DSL). El objetivo principal fue la creación de un código más intuitivo para los usuarios y con mucho más alcance y funcionalidad que los programas y aplicaciones existentes.

A pesar de las dificultades encontradas, el equipo logró superar dichos obstáculos y crear un compilador funcional capaz de generar formularios en HTML y poder almacenar sus respuestas en un Google Spreadsheet a partir de las especificaciones dadas.

Como equipo podemos afirmar que el proyecto permite poner en práctica los conceptos aprendidos a lo largo del cuatrimestre en la materia. Desde la definición del alfabeto con flex, la creación de producciones en la parte de Bison, hasta el análisis de los árboles binarios para las condiciones lógicas.

6. Referencias

1. T. Tidwall, *hashmap.c: A simple and fast hashmap implementation in C*. [En línea]. Disponible en: <https://github.com/tidwall/hashmap.c>. [Accedido: noviembre 2024].
2. *Vue.js - The Progressive JavaScript Framework*. [En línea]. Disponible en: <https://vuejs.org/>. [Accedido: noviembre 2024].
3. *Vuetify - Material Design Framework*. [En línea]. Disponible en: <https://vuetifyjs.com/>. [Accedido: noviembre 2024].

7. Bibliografía

1. *Vuetify - Component Explorer*. [En línea]. Disponible en: <https://vuetifyjs.com/en/components/explorer/>. [Accedido: noviembre 2024].
2. *Vue.js - Reactivity Fundamentals*. [En línea]. Disponible en: <https://vuejs.org/guide/essentials/reactivity-fundamentals>. [Accedido: noviembre 2024].